API TESTING · TRIPLETEN QA ENGINEER PROGRAM

API Testing with Postman: Urban Grocers

Add-to-kit and delivery-cost endpoints of a practice grocery-ordering API.

Michael Andeck · mikeandeck@gmail.com · linkedin.com/in/mandeck · github.com/mandeck/Portfolio

71	35 / 36	26	JSON + XML
test cases designed and executed	passed / failed	distinct bugs logged	request bodies sent from Postman

About the application

Urban Grocers is a practice grocery-ordering application. Its main feature is themed grocery kits, such as Picnic or Pasta, built from a catalogue of 92 items; users can also create their own kits. Orders are shipped by one of several delivery services, each with its own endpoint and operating hours. A service is available only when the order falls within its hours, and the shipping price depends on the number of products and their total weight. This project tests adding products to a kit and the Fast Delivery service (07:00 to 21:00), which returns one shipping cost for the company (host) and one for the customer (client).

Scope

The assignment was to analyze two requirements, design test cases in a spreadsheet, run them in Postman, and report bugs in Jira. The first requirement covers adding products to a kit. The second covers delivery: whether the Fast Delivery service is available at a given hour, and what it costs. Each case sends a request and compares the status code and response with the requirement and the API documentation (apiDoc).

ENDPOINT	REQUIREMENT UNDER TEST
POST /api/v1/kits/:id/products JSON body	Adds existing products to a kit. A kit holds at most 30 distinct products, and adding more is rejected. A nonexistent product ID returns 400, a nonexistent kit ID returns 404, and a wrongly structured body returns 400.
POST /fast-delivery/v3.1.1/calculate-delivery.xml XML body	Fast Delivery is available from 07:00 to 21:00 (isItPossibleToDeliver is true or false). The host cost is 3 for 1–7 items up to 2.5 kg, 6 for 8–14 items up to 6 kg, and 7 in every other case. The client cost is 0, or 9 when the maximum item count or weight is exceeded.

Environment and tools

Computer	iMac, macOS Sonoma 14.6.1 (23G93)
Tools	Postman (requests and responses), Google Sheets (test cases and results), Jira (bugs), apiDoc (API documentation served by the test server)
Method	Equivalence partitioning and boundary values; the expected status code and response recorded for every case

A training project on a practice application, completed as part of coursework. The requirements and API documentation are TripleTen course materials, summarized here in my own words. S4P numbers are reference IDs from the project's Jira board.

Test design

I wrote positive cases for each rule, then negative cases for every input the requirement constrains, with values on both sides of each limit. Cases are labeled K (add products to a kit) and D (delivery).

INPUT	VALUES TESTED
Products in a kit	29, 30 and 31 distinct products
Kit ID	6 (highest existing), 7 and 8 or more; negative, blank, decimal and text values
Product ID	92 (highest existing), 93 and 94 or more; negative, decimal and text values
Delivery time (hour)	7, 8, 20 and 21 inside the hours; 0, 6, 22 and 23 outside; 24 or more, decimal, blank, negative and text
Item count	1, 2, 6, 7 | 8, 9, 13, 14 | 15 and 16 or more, to cross each price tier; decimal, blank, 0 and text
Weight (kg)	0, 0.1, 2.4, 2.5 | 2.6, 2.7, 5.9, 6 | 6.1 and 7; blank and text

Coverage by area

ADD PRODUCTS TO A KIT (K)	CASES	TOTAL	PASSED	FAILED
Adding products and the 30-product limit	K1–K6	6	6	0
Kit ID: negative, blank, decimal, text, missing	K7–K10, K17	5	3	2
Kit existence: highest kit, unknown kits	K20–K22, K29	4	4	0
Product ID: negative, decimal, text, highest, unknown	K11–K13, K23–K25, K27	7	1	6
Quantity: negative, decimal, text	K14–K16	3	0	3
Request body: missing field, empty, malformed	K18, K19, K26, K28	4	1	3
Total		**29**	**15**	**14**

DELIVERY COST AND AVAILABILITY (D)	CASES	TOTAL	PASSED	FAILED
Delivery time inside and outside 07:00–21:00	D1–D4, D6–D9	8	4	4
Invalid delivery time: decimal, 24 or more, blank, negative, text	D5, D10–D14	6	0	6
Item count against the price tiers	D15–D24	10	8	2
Invalid item count: decimal, blank, 0, text	D25–D28	4	0	4
Weight against the price tiers	D29–D38	10	8	2
Invalid weight: blank, text	D39–D40	2	0	2
Item count and weight combined	D41–D42	2	0	2
Total		**42**	**20**	**22**

Test case workbook

Each case records its steps, request body, expected result, actual result, status and a link to its bug report. The excerpts below show cases K8–K11 and D21–D23. The bug links are cropped out.

No	Test Cases	Test Steps	Test Data (Request Body)	Expected Result	Actual Result	Status
8	The user **cannot** leave the Kit ID blank	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request by using the designated product ID in the path params and request body. 3. The user attempts to add the designated product(s) to the kit. 4. The user gets an error code on the response.	**Path params: id=""** { "productsList": [{ "id": 3, "quantity": 3 }] }	- User receives error code "404 Not Found"	- User receives error code "404 Not Found"	Passed
9	The user **cannot** put decimal numbers in the Kit ID	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request by using the designated product ID in the path params and request body. 3. The user attempts to add the designated product(s) to the kit. 4. The user gets an error code on the response.	**Path params: id=5.5** { "productsList": [{ "id": 3, "quantity": 3 }] }	- User receives error code "404 Not Found"	- User receives error code "500 Internal Server Error"	Failed
10	The user **cannot** put strings (textual data) in the Kit ID	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request by using the designated product ID in the path params and request body. 3. The user attempts to add the designated product(s) to the kit. 4. The user gets an error code on the response.	**Path params: id=six** { "productsList": [{ "id": 3, "quantity": 3 }] }	- User receives error code "404 Not Found"	- User receives error code "500 Internal Server Error"	Failed
11	The user **cannot** put negative numbers in the Product ID	1. The user selects POST request with URL + api/v1/kits/:id/products 2. The user sends a request by using the designated product ID in the path params and request body. 3. The user attempts to add the designated product(s) to the kit. 4. The user gets an error code on the response.	Path params: id=6 { "productsList": [{ **"id": -3,** "quantity": 3 }] }	- User receives error code "400 Bad Request"	- Added a quantity with no ID to the kit. - Status: 200 OK	Failed

Figure 1. Add-products cases K8–K11: blank, decimal and text kit IDs, and a negative product ID.

No	Test Cases	Test Steps	Test Data (Request Body)	Expected Result	Actual Result	Status
21	**If "productsCount" = 13**, while "productsWeight" is 2.5kg or less, and "tobeDeliveredTime" = "true, then "hostDeliveryCost" = 6, and "clientDeliveryCost" = 0	1. The server has recently been started. 2. In Postman, the user selects to POST a request to URL + /fast-delivery/v3.1.1/calculate-delivery.xml 3. The user sets the request body to be read in XML. 4. The user adds the code as directed in the "Test Data" (Request Body) 5. The user sends the request.	<InputModel> **<productsCount>13</productsCount>** <productsWeight>1.5</productsWeight> <deliveryTime>8</deliveryTime> </InputModel>	Status: 200 OK "hostDeliveryCost" = 6 "clientDeliveryCost" = 0	Status: 200 OK "hostDeliveryCost" = 6 "clientDeliveryCost" = 0	Passed
22	**If "productsCount" = 14**, while "productsWeight" is 2.5kg or less, and "tobeDeliveredTime" = "true, then "hostDeliveryCost" = 6, and "clientDeliveryCost" = 0	1. The server has recently been started. 2. In Postman, the user selects to POST a request to URL + /fast-delivery/v3.1.1/calculate-delivery.xml 3. The user sets the request body to be read in XML. 4. The user adds the code as directed in the "Test Data" (Request Body) 5. The user sends the request.	<InputModel> **<productsCount>14</productsCount>** <productsWeight>1.5</productsWeight> <deliveryTime>8</deliveryTime> </InputModel>	Status: 200 OK "hostDeliveryCost" = 6 "clientDeliveryCost" = 0	Status: 200 OK "hostDeliveryCost" = 6 "clientDeliveryCost" = 0	Passed
23	**If "productsCount" = 15**, while "productsWeight" is 2.5kg or less, and "tobeDeliveredTime" = "true, then "hostDeliveryCost" = 7, and "clientDeliveryCost" = 9	1. The server has recently been started. 2. In Postman, the user selects to POST a request to URL + /fast-delivery/v3.1.1/calculate-delivery.xml 3. The user sets the request body to be read in XML. 4. The user adds the code as directed in the "Test Data" (Request Body) 5. The user sends the request.	<InputModel> **<productsCount>15</productsCount>** <productsWeight>1.5</productsWeight> <deliveryTime>8</deliveryTime> </InputModel>	Status: 200 OK "hostDeliveryCost" = 7 "clientDeliveryCost" = 9	Status: 200 OK "hostDeliveryCost" = 6 "clientDeliveryCost" = 6	Failed

Figure 2. Delivery cases D21–D23: item counts 13, 14 and 15 against the cost table.

Results

ENDPOINT	CASES	PASSED	FAILED		BUGS
Add products to a kit	29	15	14		12
Delivery cost	42	20	22		14
Total	**71**	**35**	**36**		**26**

Green = passed, red = failed. Several failed cases can share one bug, so 36 failed cases give 26 bugs.

PATTERN ACROSS THE 26 BUGS	BUGS
Invalid input accepted with 200 OK instead of rejected with 400	15
500 Internal Server Error instead of 400 or 404	8
Wrong shipping costs above the maximum item count and weight	2
Out-of-hours request returns no isItPossibleToDeliver value	1

Bugs logged

Severity scale. Low: cosmetic, or a small departure from the written requirements. Medium: a potentially functional problem with possible negative effects, but not catastrophic. High: a core feature fails, or a serious risk such as data integrity. Ranges mark borderline cases.

BUG	CASES	REQUEST	EXPECTED	ACTUAL	SEVERITY
Add products to a kit · 12 bugs					
S4P-1	K11	Negative product ID (−3)	400 Bad Request	200 OK, item added	Medium–High
S4P-2	K24, K25, K27	Nonexistent product ID (93 or more)	400 Bad Request	200 OK, item added	Medium–High
S4P-3	K26	Empty request body	400 Bad Request	200 OK	Low–Medium
S4P-14	K9	Decimal kit ID (5.5)	404 Not Found	500 Internal Server Error	Medium
S4P-15	K10	Text kit ID	404 Not Found	500 Internal Server Error	Medium
S4P-16	K12	Decimal product ID (3.5)	400 Bad Request	500 Internal Server Error	Medium
S4P-18	K15	Decimal quantity (3.5)	400 Bad Request	500 Internal Server Error	Medium
S4P-23	K13	Text product ID	400 Bad Request	500 Internal Server Error	Medium
S4P-24	K16	Text quantity	400 Bad Request	500 Internal Server Error	Medium
S4P-25	K18	Missing product ID	400 Bad Request	500 Internal Server Error	Medium
S4P-26	K19	Missing quantity	400 Bad Request	500 Internal Server Error	Medium
S4P-27	K14	Negative quantity (−3)	400 Bad Request	200 OK, quantity subtracted	Medium–High
Delivery cost and availability (Fast Delivery, XML) · 14 bugs					
S4P-4	D12	Blank delivery time	400 Bad Request	200 OK, deliverable = true	Medium
S4P-5	D14	Text delivery time	400 Bad Request	200 OK, deliverable = true	Medium
S4P-6	D23, D24, D42	15 or more items	Host cost 7, client cost 9	Host cost 6, client cost 6	High
S4P-7	D37, D38, D41	Weight over 6 kg	Host cost 7, client cost 9	Host cost 6, client cost 6	High
S4P-8	D39	Blank weight	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-9	D40	Text weight	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-10	D25	Decimal item count (7.5)	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-11	D27	Item count of 0	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-12	D26	Blank item count	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-13	D28	Text item count	400 Bad Request	200 OK, cost calculated	Low–Medium
S4P-19	D10, D11	Delivery time 24 or more	400 Bad Request	200 OK, service name only	Low–Medium
S4P-20	D13	Negative delivery time	400 Bad Request	200 OK, service name only	Low–Medium
S4P-21	D6–D9	Delivery time outside 07:00–21:00 (0, 6, 22, 23)	deliverable = false	Service name only, no value	Medium
S4P-22	D5	Decimal delivery time	400 Bad Request	200 OK, deliverable = true	Medium

Featured bug: S4P-15

A text kit ID returns 500 instead of 404. Case K10. Severity: Medium. The server returns an unhandled error instead of answering a bad kit ID with a client error.

R-1: POST {{server URL}}/api/v1/kits/:id/products returns 500 Internal Server error instead of 404 Not Found when string values are passed in the kit id parameter

Description

When attempting to add to a Kit ID with a string (text data), user receives error code "500 Internal Server Error".

Preconditions

- The Server has already started.

Steps

- Open Postman
- Select POST, the pathway is "(server URL) + api/v1/kits/6/products"
  - (server URL) = the unique server address created when the server is started.
  - Kit ID = six
- Click "Body" → Raw → JSON
- Paste the following into the body:

```
{
  "productsList": [
    {
      "id": 3,
      "quantity": 3
    }
  ]
}
```

- The user sends the POST request by using the designated product ID in the path params (kit ID = six) and the above written request body.

Expected Results

- User receives error code "404 Not Found"

Figure 3. Bug report S4P-15 as filed in Jira (first part).

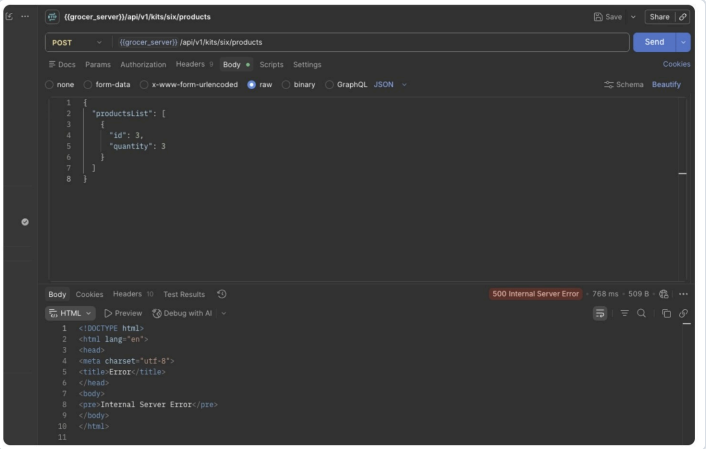


Figure 4. Postman: POST /api/v1/kits/six/products returns 500 Internal Server Error.

Request	POST /api/v1/kits/six/products with one product (ID 3, quantity 3)
Expected	404 Not Found
Actual	500 Internal Server Error
Traceability	Requirement → case K10 → bug S4P-15 → reference link recorded in the workbook

Featured bug: S4P-25

A product with no ID returns 500 instead of 400. Case K18. Severity: Medium.

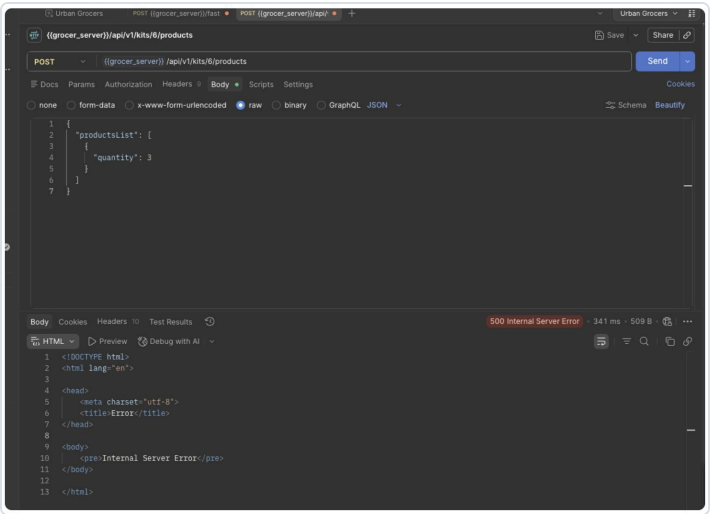


Figure 5. Postman: the request body has a quantity but no product ID.

Request	POST /api/v1/kits/6/products with a quantity of 3 and no product ID
Expected	400 Bad Request
Actual	500 Internal Server Error
Same cause	S4P-26 (missing quantity)

Featured bug: S4P-6

Orders above the maximum item count are charged the wrong shipping costs. Cases D23, D24 and D42. Severity: High.

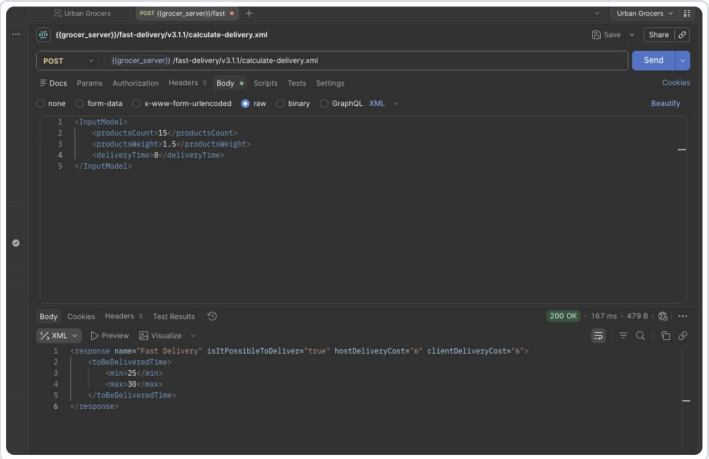


Figure 6. Postman: 15 items, 1.5 kg, delivery hour 8 returns host cost 6 and client cost 6.

Request	POST /fast-delivery/v3.1.1/calculate-delivery.xml with productsCount 15, productsWeight 1.5, deliveryTime 8
Expected	200 OK, hostDeliveryCost 7, clientDeliveryCost 9 (limit exceeded)
Actual	200 OK, hostDeliveryCost 6, clientDeliveryCost 6
Same cause	S4P-7: weight over 6 kg

Most important bugs

1. **S4P-1, S4P-2 and S4P-27, invalid data changes a kit (Medium–High).** Negative and nonexistent product IDs are accepted, and a negative quantity subtracts from the kit, instead of the request being rejected.
2. **S4P-6 and S4P-7, wrong shipping costs (High).** Orders above the maximum count or weight get host cost 6 and client cost 6, not the documented 7 and 9.
3. **Eight bugs return 500 Internal Server Error (Medium).** Decimal or text IDs and quantities, and a missing product ID or quantity, return an unhandled server error instead of a client error.